Complete project report and end-to-end workflow

This is the project methodology you should understand well enough to reproduce and explain, not just list on your resume.

Phase 1 — Download and understand the dataset

Start with the exact Sample Superstore workbook or dataset version you used.

Recommended source documentation:

- Record the original download URL.
- Save the untouched source files in `data/raw/`.
- Record the worksheet names and column names.
- Document the row count of each table before transformations.
- Record the date range and key fields.

Your source may contain separate `Orders`, `People`, and `Returns` worksheets. Confirm this from your actual workbook rather than assuming every version contains the same fields.

Initial data dictionary

Field	Business meaning	Validation
Order ID	Identifies an order	Can repeat across product lines
Order Date	Date the order was placed	Valid date; check date range
Ship Date	Date the order was shipped	Valid date; check against order date
Customer ID	Identifies a customer	Consistent customer attributes
Product ID	Identifies a product	Consistent product attributes
Sales	Revenue recorded for a line	Numeric; inspect missing/negative values
Profit	Profit recorded for a line	Numeric; negative values may be legitimate
Quantity	Units sold	Numeric; check for invalid values
Discount	Applied discount	Check range and interpretation
Region	Sales geography	Standardized category values
Returned	Order return indicator	Confirm the source and merge logic

Phase 1 output

- Original dataset.
- Data dictionary.
- Initial row counts.
- Data quality notes.

Phase 2 — Excel data inspection and validation

Excel is useful for quickly understanding a dataset before writing complex code.

Recommended procedure

1. Open the original workbook and inspect every worksheet.
2. Convert the data range to an Excel Table where appropriate.
3. Review data types and date formats.
4. Use filters to find blanks and unusual values.
5. Use Remove Duplicates only when the selected business key proves the records are duplicates.
6. Use PivotTables to summarize sales and profit by category and region.
7. Compare the summaries with later SQL and Power BI outputs.

Useful Excel tools include `COUNTIF`, `SUMIFS`, `COUNTIFS`, PivotTables, conditional formatting, and Power Query.

For example, a PivotTable with Category as rows and Sales and Profit as values gives you a fast initial view of business performance.

Critical lesson: Don't remove every repeated Order ID. Multiple rows can represent different products or line items belonging to one order.

Phase 3 — MySQL database and analytical SQL

Step 1: Create the database

```
CREATE DATABASE superstore_analytics;
USE superstore_analytics;
```

Create tables that match the source columns and their data types. Preserve identifier fields carefully, especially if any IDs contain leading zeros.

Step 2: Load the data

You can import the workbook after exporting its worksheets to CSV, or use an appropriate import process for your database environment.

After loading, verify that the row counts match the source.

```
SELECT COUNT(*) AS row_count
FROM orders;
```

Run equivalent checks for each imported table, adjusting table names to your schema.

Step 3: Perform data quality checks

Example: find orders with missing identifiers.

```
SELECT COUNT(*) AS missing_order_ids
FROM orders
```

```
WHERE `Order ID` IS NULL;
```

Example: inspect repeated order identifiers.

```
SELECT
    `Order ID`,
    COUNT(*) AS line_count
FROM orders
GROUP BY `Order ID`
HAVING COUNT(*) > 1
ORDER BY line_count DESC;
```

This query identifies orders with multiple lines. It does not prove that those rows are erroneous duplicates.

Example: verify the sales and profit totals.

```
SELECT
    SUM(Sales) AS total_sales,
    SUM(Profit) AS total_profit,
    SUM(Profit) / NULLIF(SUM(Sales), 0) AS profit_margin
FROM orders;
```

Use your actual table and column names. Quote identifiers if they contain spaces or reserved words.

Step 4: Perform business analysis

Organize your SQL scripts by business question.

A. Overall performance

Total sales, profit, margin, distinct orders, and quantity.

B. Category and product analysis

Sales and profit by category and sub-category, top products, and loss-making products.

C. Customer and region analysis

Segment contribution, regional comparison, and customer-level sales.

D. Time analysis

Monthly performance, growth comparisons, and cumulative trends.

E. Advanced SQL

CTEs, subqueries, window functions, ranking, running totals, and query optimization where applicable.

For example, monthly sales:

```
SELECT
    DATE_FORMAT(`Order Date`, '%Y-%m') AS year_month,
    SUM(Sales) AS total_sales,
    SUM(Profit) AS total_profit,
    SUM(Profit) / NULLIF(SUM(Sales), 0) AS profit_margin
FROM orders
GROUP BY DATE_FORMAT(`Order Date`, '%Y-%m')
ORDER BY year_month;
```

If your table uses a different date field or schema, adjust the query accordingly.

Step 5: Join return information safely

The `Returns` table may contain one record per order, but you must verify its grain before joining it to the sales line table.

If returns are order-level, joining them to multiple sales lines can repeat the return flag across those lines. That is acceptable for identifying returned orders, but it becomes dangerous if you sum an order-level amount after a many-to-many join.

Validate that the return key is unique before joining. Never join tables on a weak or incorrect key just to make the query run.

Phase 3 output

- Database schema and imported tables.
- SQL data quality checks.
- Analytical query scripts.
- Exported results, where needed.
- Validated aggregate totals.

Phase 4 — Python exploratory data analysis

Your Python stage should provide reproducible analysis and independently validate important SQL findings.

Step 1: Set up the environment

Create a virtual environment if you want a reproducible project setup.

Example `requirements.txt`:

```
pandas
numpy
matplotlib
seaborn
jupyter
openpyxl
```

If you connect directly to MySQL from Python, include the database connector you actually use. If you only read CSV exports, a database connector is not required.

Step 2: Load and inspect the data

```
import pandas as pdimport numpy as npimport matplotlib.pyplot as pltimport
seaborn as snsorders = pd.read_csv(
"../data/raw/orders.csv")print("Shape:",
orders.shape)print(orders.head())print(orders.info())print(orders.isna().su
m())print("Exact duplicate rows:", orders.duplicated().sum())
```

Adjust the filename and path to match your project. If your source is Excel, `pd.read_excel()` may be more appropriate.

Step 3: Clean and validate

```
orders["Order Date"] = pd.to_datetime(    orders["Order Date"],
errors="coerce")orders["Ship Date"] = pd.to_datetime(    orders["Ship
Date"],    errors="coerce")orders["Sales"] = pd.to_numeric(
orders["Sales"],    errors="coerce")orders["Profit"] = pd.to_numeric(
orders["Profit"],    errors="coerce")
```

After conversion, inspect how many values became missing. Don't silently drop all missing rows without understanding their impact.

Step 4: Calculate core metrics

```
total_sales = orders["Sales"].sum()total_profit =
orders["Profit"].sum()profit_margin = (    total_profit / total_sales    if
total_sales != 0    else np.nan)total_orders = orders["Order
ID"].nunique()total_quantity = orders["Quantity"].sum()print("Total
sales:", total_sales)print("Total profit:", total_profit)print("Profit
margin:", profit_margin)print("Distinct orders:", total_orders)print("Total
quantity:", total_quantity)
```

The output should be compared with MySQL and Power BI under equivalent filters and definitions.

Step 5: Explore category and regional performance

```
category_summary = (    orders.groupby("Category", dropna=False)    .agg(
Sales=("Sales", "sum"),        Profit=("Profit", "sum"),
Quantity=("Quantity", "sum"),        Orders=("Order ID", "nunique")    )
.reset_index())category_summary["Profit Margin"] = (
category_summary["Profit"]    / category_summary["Sales"].replace(0,
np.nan))print(category_summary.sort_values("Sales", ascending=False))
```

Repeat the analysis by sub-category, region, segment, and month.

Step 6: Build visualizations

```
monthly = (    orders.assign(        YearMonth=orders["Order
Date"].dt.to_period("M")    )    .groupby("YearMonth")    .agg(
Sales=("Sales", "sum"),        Profit=("Profit", "sum")    )
.reset_index())monthly["YearMonth"] =
monthly["YearMonth"].astype(str)plt.figure(figsize=(12,
5))plt.plot(monthly["YearMonth"], monthly["Sales"],
label="Sales")plt.plot(monthly["YearMonth"], monthly["Profit"],
```

```
label="Profit")plt.xticks(rotation=60)plt.title("Monthly Sales and
Profit")plt.xlabel("Month")plt.ylabel("Amount")plt.legend()plt.tight_layout
()plt.show()
```

This creates an exploratory monthly trend. You can improve it by using separate axes or separate charts when the scale difference makes one series difficult to read.

Phase 4 output

- Jupyter notebook.
- Reproducible calculations.
- Exploratory charts.
- Data quality findings.
- SQL-to-Python reconciliation.

Phase 5 — Power BI transformation and modeling

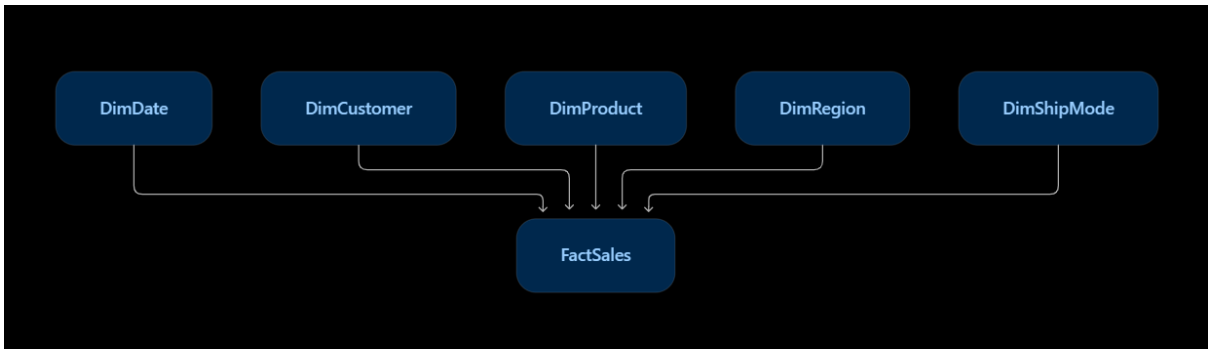
This is where your project becomes an interactive reporting solution.

Step 1: Import the data

Load the relevant CSV or Excel tables into Power BI Desktop. Use Power Query Editor to inspect data types, clean text, handle nulls, and verify key fields.

Step 2: Build the model

A recommended dimensional structure is:



`FactSales` contains line-level sales data. The dimensions provide descriptive attributes for filtering and grouping.

A separate Returns staging query can be used to merge a returned flag into the fact table, after confirming that the return key is unique.

Use one-to-many relationships from dimensions to the fact table and verify the filter direction and active relationships. Your exact model may include additional dimensions.

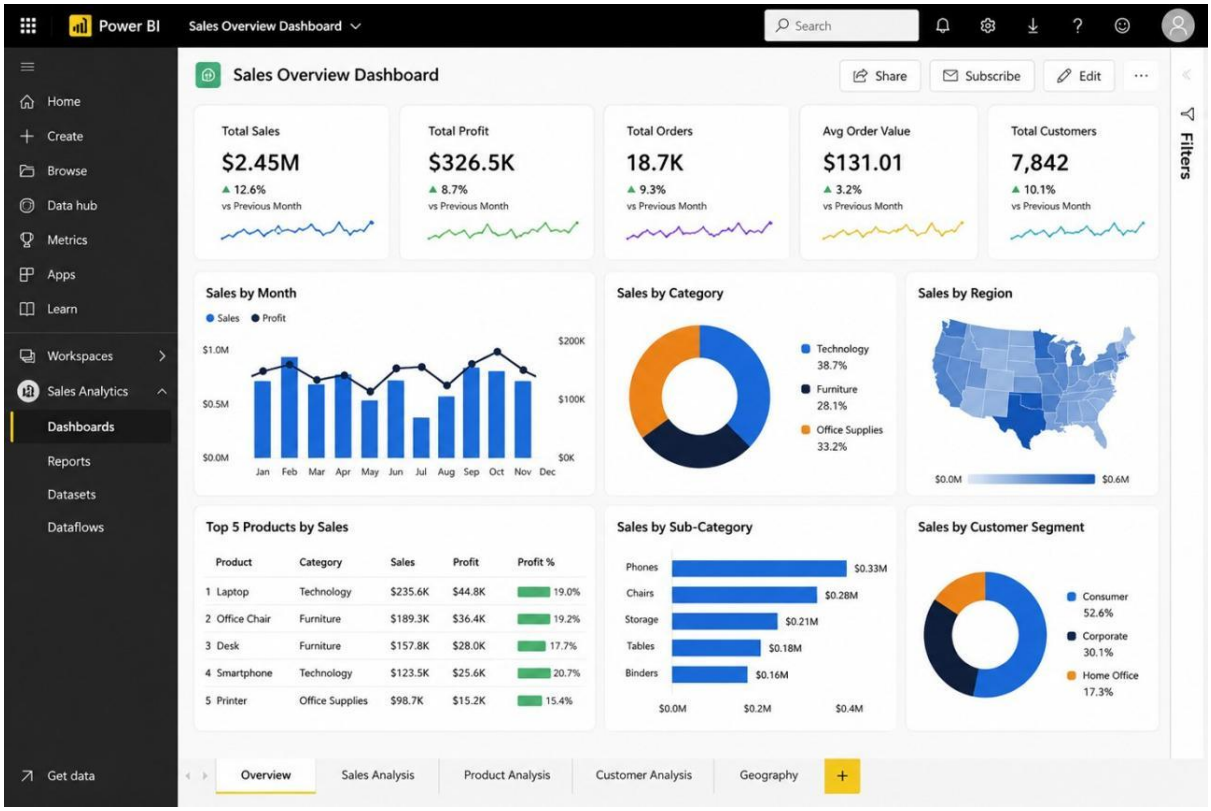
Step 3: Create the core DAX measures

You already have these seven core measures:

- Total Sales
- Total Profit
- Profit Margin
- Total Orders
- Total Quantity
- Returned Orders
- Return Rate

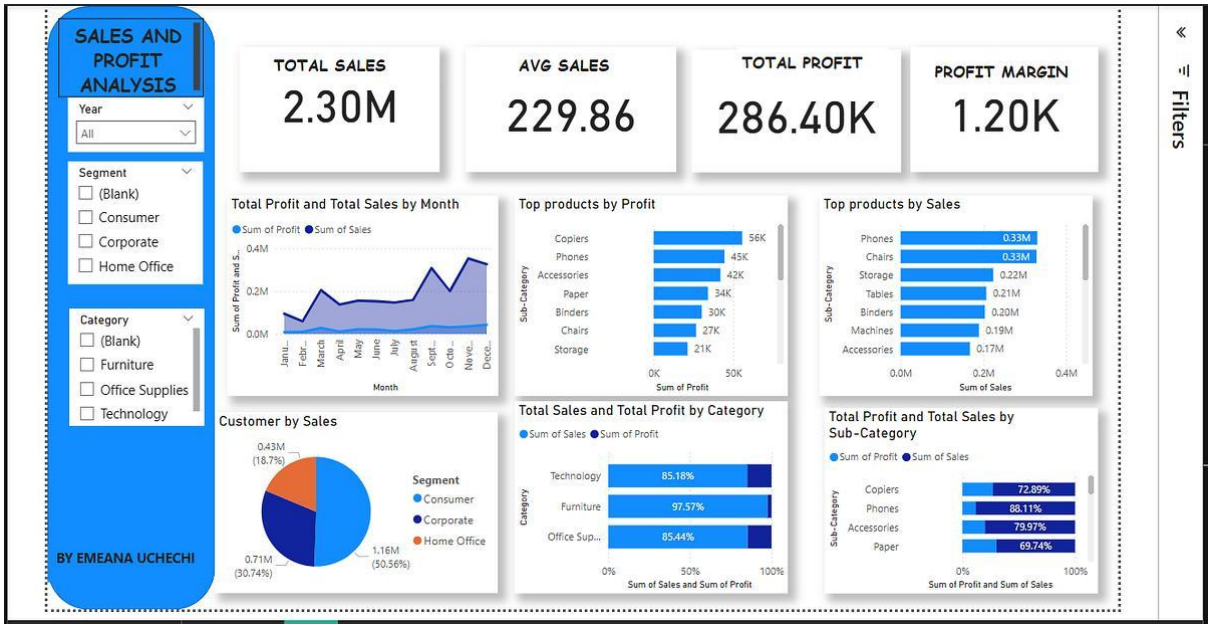
Keep them in a dedicated measures table if you prefer an organized model. The measures should be tested with slicers and against independent SQL results.

Step 4: Build the report pages



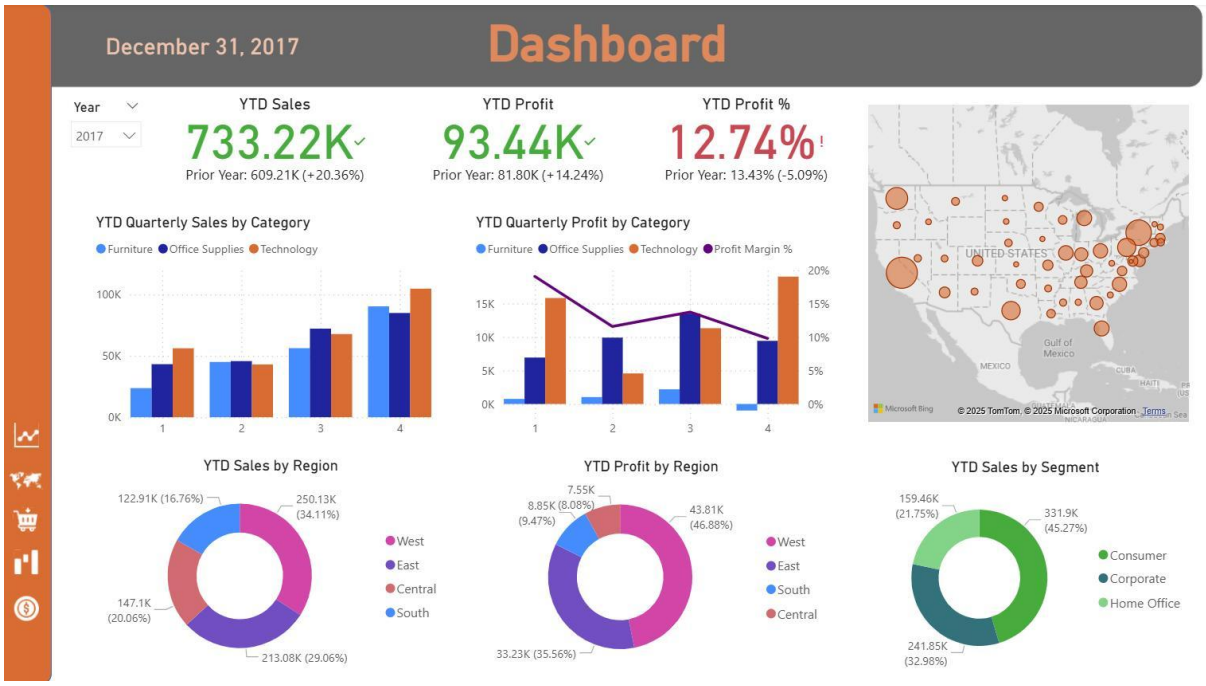
Page 1 — Executive Overview

KPIs, monthly sales and profit, category performance, regional comparison, and global slicers.



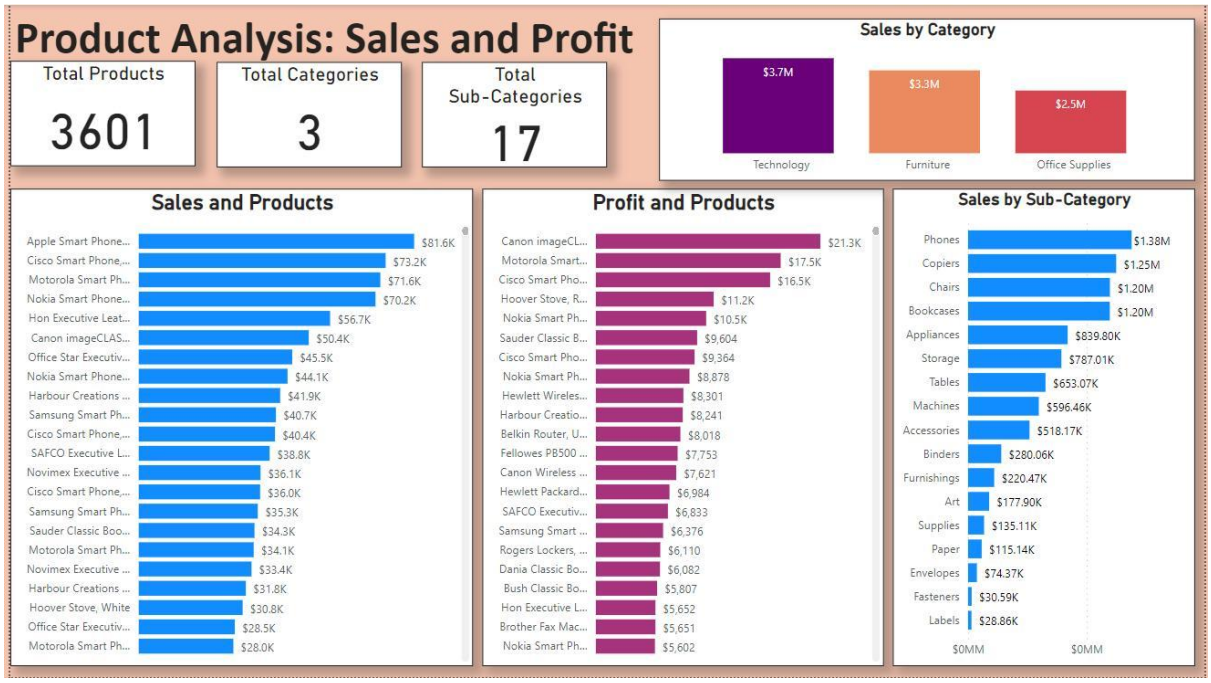
Page 2 — Category Analysis

Category and sub-category sales, profit, margin, and loss-making analysis.



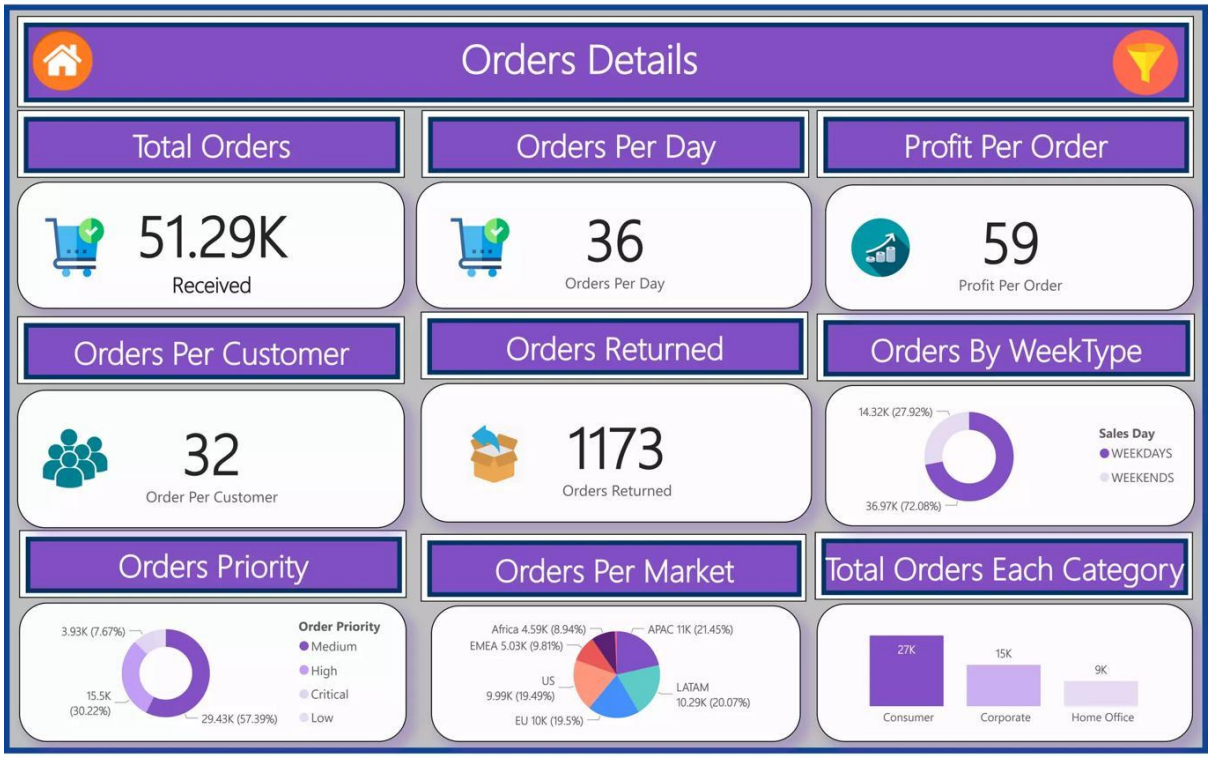
Page 3 — Regional Analysis

Sales and profit by region, regional margins, and state-level analysis if geographic fields are available.



Page 4 — Product Detail

Product ranking, sales, profit, margin, and sub-category comparisons.



Page 5 — Order Details

Transaction table, order status and return information, and order-level drill-through.

These images are visual design references, not screenshots of your actual data. Use the figures calculated by your model.

Step 5: Add advanced analysis

Your subsequent Power BI lessons included:

- MoM and YoY growth.
- YTD sales and profit.
- Target achievement and variance.
- Conditional formatting.
- Tooltips, drill-down, and drill-through.
- KPI cards and actual-versus-target visuals.

Use these only where the measures and target data have been correctly implemented.

Phase 5 output

- Power BI data model.
- DAX measures.
- Interactive report pages.
- Visual and filter validation.

Phase 6 — Publish and validate

1. Save your final `.pbix` file.
2. Sign in to Power BI Service.
3. Publish to an available workspace.
4. Open the report online and check its visuals.
5. Review the semantic model and data-source settings.
6. Configure credentials and refresh if required.
7. Verify sharing permissions and RLS if the report will be used by other people.

For local CSV files, refresh in the Service may require a gateway or a supported cloud source configuration. Publishing does not automatically guarantee future refreshes.

Phase 7 — Validate the complete pipeline

The final validation should compare equivalent metrics across the tools.

Metric	Excel	MySQL	Python	Power BI
Total Sales	PivotTable sum	`SUM(Sales)`	`Sales.sum()`	`[Total Sales]`
Total Profit	PivotTable sum	`SUM(Profit)`	`Profit.sum()`	`[Total Profit]`
Profit Margin	Profit ÷ Sales	Profit ÷ Sales	Profit ÷ Sales	`[Profit Margin]`
Distinct Orders	Distinct count	`COUNT(DISTINCT ...)`	`nunique()`	`[Total Orders]`

Returned Orders	Validated return count	Distinct returned IDs	Distinct returned IDs	`[Returned Orders]`

If values disagree, check row counts, date filters, join duplication, distinct-count definitions, and return logic before concluding that one tool is wrong.